Capítulo 2: Fundamentos de Git

Actividad 2 - Basado en Pro Git (Chacon & Straub, 2014) - Juan Manuel Gomez Perez - 43972

Introducción

Este capítulo cubre los comandos esenciales del uso diario de Git: obtener un repositorio, registrar cambios, consultar el historial, deshacer errores, trabajar con repositorios remotos, usar etiquetas y crear alias.

2.1 Obtener un repositorio

Hay dos formas de obtener un repositorio: inicializar uno en una carpeta existente o clonar uno que ya existe en un servidor.

```
git init
git clone https://github.com/usuario/repositorio.git
git clone <url> mi-carpeta
```

2.2 Guardar cambios en el repositorio

Cada archivo del directorio de trabajo está rastreado (tracked) o sin rastrear (untracked). Los rastreados pueden estar sin modificar, modificados o preparados. El comando git status muestra en qué estado está cada archivo.

```
git status
git status -s
git add archivo.txt
git add .
```

Ignorar archivos

Los archivos que no deben rastrearse (compilados, logs, credenciales) se listan en un archivo .gitignore, un patrón por línea.

```
*.log
build/
.env
```

Ver cambios

```
git diff          # cambios no preparados
git diff --staged   # cambios preparados
```

Confirmar cambios

```
git commit
git commit -m "Mensaje claro y descriptivo"
git commit -a -m "Mensaje"   # omite el git add de archivos ya rastreados
```

Eliminar y mover archivos

```
git rm archivo.txt
git rm --cached archivo.txt   # deja de rastrearlo pero lo conserva en disco
git mv origen.txt destino.txt
```

2.3 Ver el historial de confirmaciones

```
git log
git log --oneline
git log -p -2          # muestra las diferencias de las últimas 2 confirmaciones
git log --stat
git log --graph --oneline --all
git log --since="2 weeks ago"
git log --author="nombre"
```

El comando git log admite muchos filtros y formatos, lo que permite encontrar rápidamente quién cambió qué y cuándo.

2.4 Deshacer cosas

Algunas acciones de deshacer no se pueden revertir, así que conviene tener cuidado con los comandos que descartan cambios locales.

```
git commit --amend             # corrige la última confirmación
git restore --staged archivo.txt # saca un archivo del área de preparación
git restore archivo.txt        # descarta cambios locales (¡no se puede recuperar!)
```

2.5 Trabajar con remotos

```
git remote -v
git remote add origin <url>
git fetch origin
git pull
git push origin main
git remote show origin
git remote rename origin nuevo
git remote remove origin
```

git fetch descarga los datos del remoto sin fusionarlos. git pull descarga y además intenta fusionar con la rama actual. git push envía las confirmaciones locales al servidor.

2.6 Etiquetas

Las etiquetas (tags) marcan puntos importantes del historial, normalmente versiones publicadas (v1.0, v2.0).

```
git tag
git tag -a v1.0 -m "Versión 1.0"
git show v1.0
git push origin v1.0
```

```
git push origin --tags
git checkout v1.0
```

Existen etiquetas ligeras (solo un puntero a una confirmación) y anotadas (con autor, fecha y mensaje); las anotadas son las recomendadas.

2.7 Alias de Git

```
git config --global alias.co checkout
git config --global alias.st status
git config --global alias.last "log -1 HEAD"
```

Los alias permiten abreviar comandos largos o crear atajos personalizados.

Fuente: Chacon, S., & Straub, B. (2014). Pro Git (2.ª ed.). Apress.
https://git-scm.com/book/es/v2